

CHƯƠNG 23: HỌC TANG CƯỜNG (REINFORCEMENT LEARNING)

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 23

- ◇ 23.1 Học từ các Phần thưởng (Learning from Rewards)
- ◇ 23.2 Học tăng cường Thụ động (Passive RL)
- ◇ 23.3 Học tăng cường Chủ động (Active RL)
- ◇ 23.4 Khái quát hóa trong RL (Generalization)
- ◇ 23.5 Tìm kiếm chính sách (Policy Search)
- ◇ 23.6 Học tăng cường Ngược (Inverse RL)
- ◇ 23.7 Các Ứng dụng Thực tế

23.1 Học từ các Phần thưởng

◇ **Học tăng cường (RL):** Đưa AI ra ngoài thế giới thực, nơi không có ai gán nhãn dữ liệu sẵn cho nó.

◇ **Cơ chế tương tác cơ bản:**

- **Tác nhân (Agent)** ở trạng thái s thực hiện một hành động a .
- **Môi trường (Environment)** thay đổi sang trạng thái s' và trả về một **Phần thưởng (Reward - R)** (có thể âm hoặc dương).

◇ **Mục tiêu tối thượng:** Tác nhân phải tự học ra một **Chính sách (Policy - π)** để quyết định nên làm gì ở mọi trạng thái nhằm *Tối đa hóa tổng phần thưởng tích lũy*.

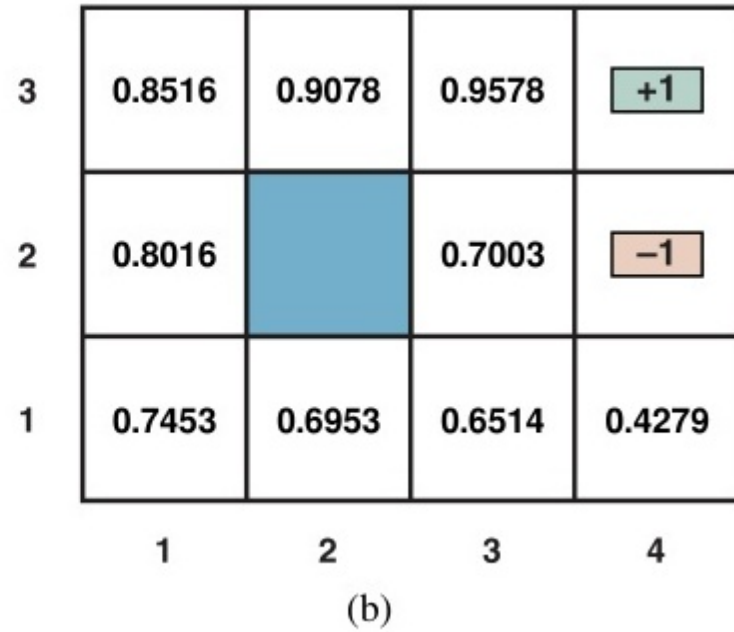
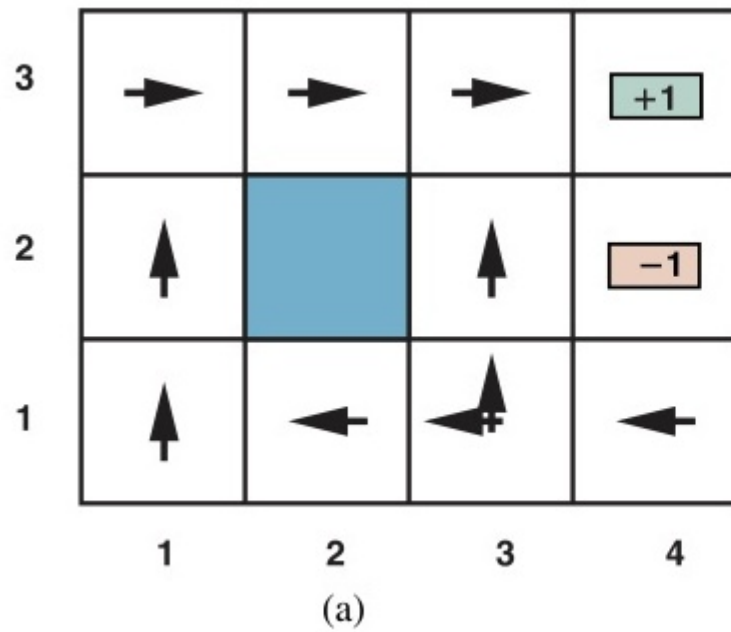


Figure 1: Môi trường Lưới 4×3 và một chính sách tối ưu để đi tới đích đạt điểm +1 trong Học tăng cường.

23.2 Học tăng cường thụ động (Passive RL)

◇ **Giả định:** Tác nhân ĐÃ CÓ một chính sách cố định $\pi(s)$. Nó ngoan ngoãn làm theo mọi hành động mà π chỉ định.

◇ **Nhiệm vụ (Policy Evaluation):** Tác nhân chỉ quan sát và học cách Đánh giá xem chính sách này mang lại độ thỏa dụng $U^\pi(s)$ là bao nhiêu cho mỗi trạng thái.

◇ **Ước lượng độ thỏa dụng trực tiếp:**

- Tác nhân chơi qua nhiều tập (Episodes). Ghi lại tổng phần thưởng nhận được từ trạng thái s cho đến hết game.
- Sau hàng vạn lần chơi, lấy Trung bình cộng để ra $U(s)$. Rất dễ nhưng bỏ qua tính liên kết của môi trường.

23.2 Quy hoạch động thích ứng (ADP)

◇ **Adaptive Dynamic Programming (ADP):** Khôn ngoan hơn phương pháp trực tiếp.

◇ Tác nhân vừa chơi vừa học **Mô hình chuyển đổi** $P(s'|s, a)$ và **Hàm phần thưởng** $R(s)$.

◇ Khi đã nắm được quy luật của thế giới (Môi trường), nó dùng ngay ****Phương trình Bellman**** để giải ra toàn bộ độ thỏa dụng:

$$U^\pi(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) U^\pi(s')$$

◇ Trong đó γ là ****Hệ số chiết khấu (Discount factor)****, giúp tác nhân coi trọng phần thưởng trước mắt hơn là tương lai xa xôi.

function PASSIVE-ADP-LEARNER(*percept*) **returns** an action

inputs: *percept*, a percept indicating the current state s' and reward signal r

persistent: π , a fixed policy
 mdp , an MDP with model P , rewards R , actions A , discount γ
 U , a table of utilities for states, initially empty
 $N_{s'|s,a}$, a table of outcome count vectors indexed by state and action, initially zero
 s, a , the previous state and action, initially null

if s' is new **then** $U[s'] \leftarrow 0$

if s is not null **then**
 increment $N_{s'|s,a}[s, a][s']$
 $R[s, a, s'] \leftarrow r$
 add a to $A[s]$
 $\mathbf{P}(\cdot \mid s, a) \leftarrow \text{NORMALIZE}(N_{s'|s,a}[s, a])$
 $U \leftarrow \text{POLICYEVALUATION}(\pi, U, mdp)$
 $s, a \leftarrow s', \pi[s']$

return a

Figure 2: Sơ đồ Tác tử học tăng cường thụ động bằng Quy hoạch động thích ứng (ADP).

23.2 Học theo Chênh lệch Thời gian (TD)

◇ ADP đòi hỏi phải giải một hệ phương trình tuyến tính khổng lồ. Rất tốn kém!

◇ **Temporal-Difference Learning (TD):** Học trực tiếp từ Trải nghiệm thực tế mà không cần mô hình $P(s'|s, a)$.

◇ Nếu trạng thái s dẫn đến s' , thì $U(s)$ phải "đồng điệu" với $U(s')$.

◇ **Phương trình cập nhật TD:**

$$U(s) \leftarrow U(s) + \alpha (R(s) + \gamma U(s') - U(s))$$

- α là **Tốc độ học (Learning rate)**.
- Sự chênh lệch $R(s) + \gamma U(s') - U(s)$ chính là "Sai số thời gian" (TD Error).

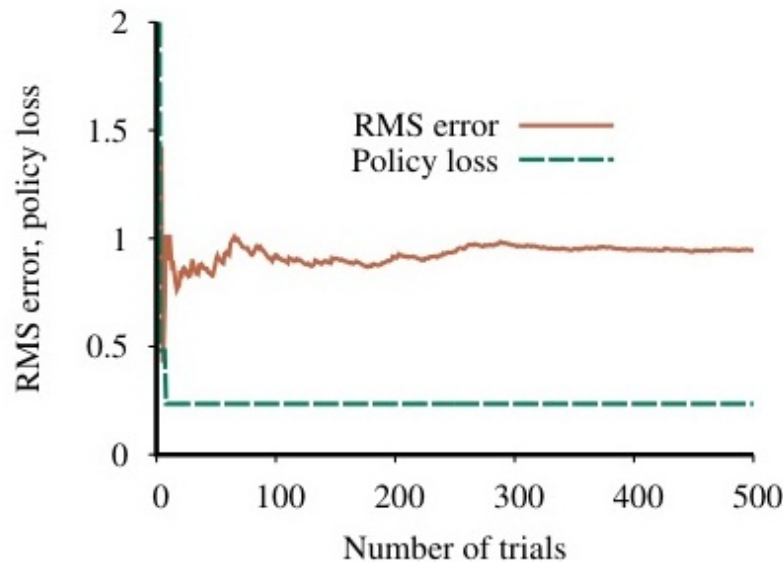
function PASSIVE-TD-LEARNER(*percept*) **returns** an action
inputs: *percept*, a percept indicating the current state s' and reward signal r
persistent: π , a fixed policy
 s , the previous state, initially null
 U , a table of utilities for states, initially empty
 N_s , a table of frequencies for states, initially zero

if s' is new **then** $U[s'] \leftarrow 0$
if s is not null **then**
 increment $N_s[s]$
 $U[s] \leftarrow U[s] + \alpha(N_s[s]) \times (r + \gamma U[s'] - U[s])$
 $s \leftarrow s'$
return $\pi[s']$

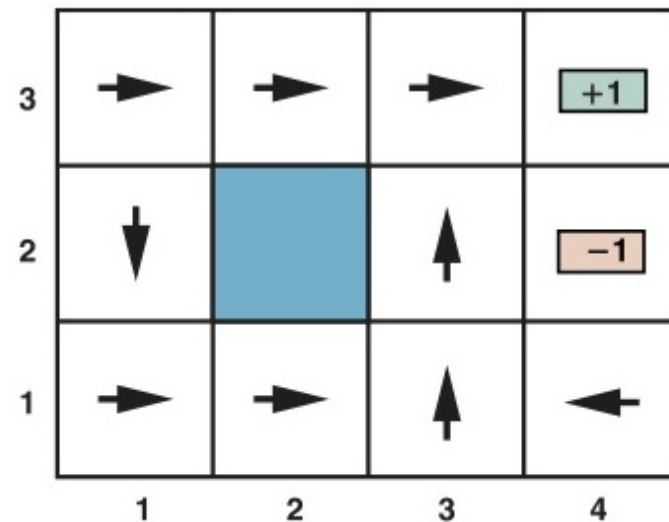
Figure 3: Thuật toán Học theo Chênh lệch Thời gian (TD Learning).

23.3 Học tăng cường chủ động (Active RL)

- ◇ Bây giờ, Tác nhân KHÔNG CÓ chính sách nào cả. Nó rơi vào thế giới với cái đầu trống rỗng.
- ◇ Nó phải tự mình thử nghiệm mọi hành động a ở mọi trạng thái s để tìm ra ****Chính sách Tối ưu π^{**}** .



(a)



(b)

Figure 4: Hiệu suất của một tác tử tham lam (greedy) bị kẹt ở cực đại cục bộ do thiếu Khám phá (Exploration).

◇ Nó không chỉ cần học $U(s)$, mà còn phải tính được độ tốt của từng Cặp (Trạng thái, Hành động).

23.3 Khám phá vs Khai thác

- ◇ ****Trái tim của AI tự học:**** Bài toán Exploration vs Exploitation.
- ◇ **Khai thác (Exploitation):** Tác nhân luôn chọn hành động mang lại phần thưởng cao nhất theo những gì nó *đã biết*.
 - Rủi ro: Bị kẹt ở cực đại cục bộ.
- ◇ **Khám phá (Exploration):** Tác nhân chọn những hành động nó chưa từng thử, chấp nhận rủi ro bị phạt.
 - Hy vọng tìm ra một con đường mới dẫn đến kho báu lớn hơn nhiều.
- ◇ **Khám phá an toàn (Safe Exploration):** Kỹ thuật giúp tác nhân thử nghiệm nhưng tránh rơi xuống vực thẳm.

23.3 Thuật toán Q-Learning

- ◇ Là đỉnh cao của Học tăng cường "Phi mô hình" (Model-free).
- ◇ Thay vì học $U(s)$, tác nhân học **Hàm $Q(s, a)$ **: Giá trị của việc thực hiện hành động a khi đang ở trạng thái s .

◇ **Phương trình cập nhật Q-Learning:**

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(R(s) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

- ◇ Điều kỳ diệu của Q-Learning: Tác nhân hoàn toàn **không cần biết** xác suất chuyển đổi $P(s'|s, a)$ của môi trường, nó chỉ cần chạy loanh quanh và áp dụng công thức trên!

function Q-LEARNING-AGENT(*percept*) **returns** an action
inputs: *percept*, a percept indicating the current state s' and reward signal r
persistent: Q , a table of action values indexed by state and action, initially zero
 N_{sa} , a table of frequencies for state–action pairs, initially zero
 s, a , the previous state and action, initially null

if s is not null **then**
 increment $N_{sa}[s, a]$
 $Q[s, a] \leftarrow Q[s, a] + \alpha(N_{sa}[s, a])(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$
 $s, a \leftarrow s', \operatorname{argmax}_{a'} f(Q[s', a'], N_{sa}[s', a'])$
return a

Figure 5: Tác tử Q-learning chủ động khám phá môi trường để tối ưu hóa Hàm Giá trị $Q(s, a)$.

23.4 Khái quát hóa trong RL

◇ Nếu không gian trạng thái nhỏ (mê cung 4×4), ta lưu Q-value trong một cái Bảng (Table).

◇ Nhưng Cờ Vua có 10^{43} trạng thái. Cờ Vây có 10^{170} trạng thái. Không bộ nhớ nào chứa nổi một cái bảng khổng lồ như vậy.

◇ **Giải pháp - Xấp xỉ Hàm (Function Approximation):**

- Sử dụng một Hàm (ví dụ: Đa thức hoặc Mạng Nơ-ron) với tập tham số θ để biểu diễn $\hat{U}_\theta(s)$ hoặc $\hat{Q}_\theta(s, a)$.
- Nếu tác nhân học được giá trị của 1 trạng thái, mạng Nơ-ron sẽ **Khái quát hóa** cho hàng triệu trạng thái tương tự (mà không cần thử lại).

23.4 Học tăng cường sâu (Deep RL)

◇ Khi kết hợp Q-Learning với mạng Nơ-ron sâu (Deep Learning), ta có **Deep Q-Networks (DQN)**.

◇ **Kỷ nguyên mới của AI:**

- Đầu vào của mạng Nơ-ron chính là Pixel màn hình của một trò chơi.
- Đầu ra là giá trị Q cho nút Bấm (Lên, Xuống, Bắn, Nhảy).
- Năm 2013, DeepMind đã dùng DQN để dạy AI chơi mọi trò chơi Atari của con người với trình độ siêu phàm mà chỉ cần nhìn màn hình như con người.

23.5 Tìm kiếm chính sách (Policy Search)

◇ Có những bài toán, việc tính hàm Q rất khó (do hành động là liên tục, ví dụ: Xoay vô lăng bao nhiêu độ).

◇ **Policy Search:** Thay vì học giá trị $Q(s, a)$, ta thiết kế mạng Nơ-ron học **Trực tiếp** một chính sách $\pi_\theta(s, a)$.

- Mạng sẽ nhận đầu vào là Cảm biến (Sensor).
- Đầu ra là *Xác suất* thực hiện một hành động (Ví dụ: 80% sang trái, 20% sang phải).

◇ ****Policy Gradient:**** Điều chỉnh các tham số θ sao cho xác suất của những hành động mang lại phần thưởng cao sẽ TANG LÊN.

23.6 Học tăng cường Ngược (IRL)

◇ Trong thực tế, thiết kế Hàm phần thưởng $R(s)$ cực kỳ gian nan. (Ví dụ: Dạy ô tô lái xe an toàn, nếu chỉ thưởng vì đến đích nhanh thì nó sẽ đâm người).

◇ Học Việc (Apprenticeship Learning):

- Cho AI quan sát một **Chuyên gia con người** đang làm việc.

◇ Inverse Reinforcement Learning (IRL):

- Từ những hành động của chuyên gia, AI cố gắng **Đảo ngược quy trình** để tìm ra Hàm Phần Thưởng ẩn giấu bên trong đầu chuyên gia đó.
- Sau đó, AI dùng RL thông thường để tối ưu theo Hàm phần thưởng vừa tìm được.

23.7 Các Ứng dụng Thực tế

◇ Trò chơi (Game playing):

- **TD-Gammon:** Nhà vô địch Cờ thỏ cáo đầu tiên do AI tự chơi với chính nó hàng triệu ván.
- **AlphaGo (Google DeepMind):** Hạ gục nhà vô địch cờ vây thế giới Lee Sedol bằng cách kết hợp Deep RL, Mạng CNN và Tìm kiếm cây Monte Carlo (MCTS).

23.7 Điều khiển Robot

- ◇ RL là trái tim của Robot học hành vi.
- ◇ **Trục thăng lộn ngược (Autonomous Helicopter):**
 - Stanford đã đào tạo một chiếc trục thăng nhào lộn trên không - điều mà ngay cả phi công kỳ cựu cũng rất khó thực hiện, dựa vào IRL và Policy Search.
- ◇ **Xe Tự Hành (Autonomous Vehicles):**
 - Xe tự lái sử dụng DRL trong giả lập để học cách tránh chướng ngại vật trước khi triển khai ngoài đời thực.

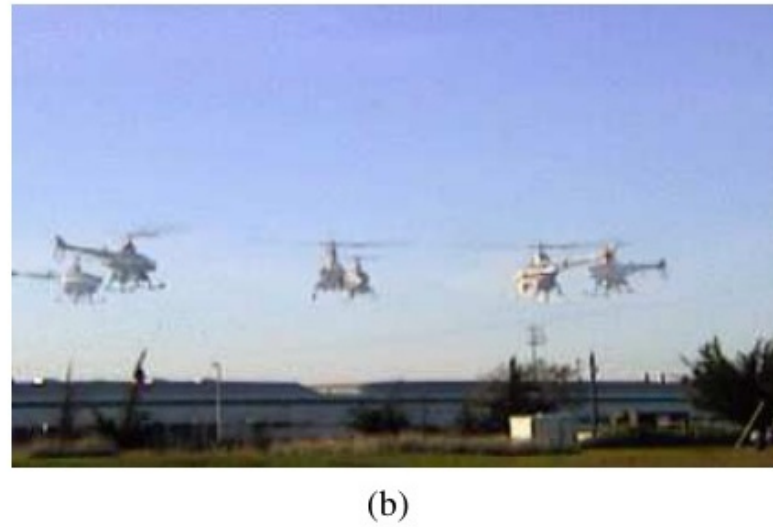
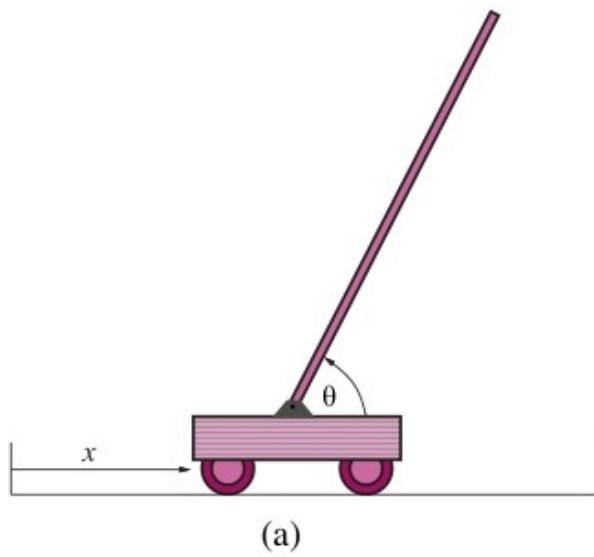


Figure 6: Thiết lập cho bài toán kinh điển Cartpole (Cân bằng cột trên xe kéo di chuyển) trong điều khiển Robot bằng Học tăng cường.

Tóm tắt Chương 23

- ◇ **RL Thụ động:** Dùng TD Learning hoặc Phương trình Bellman để đánh giá độ tốt của một chính sách cho sẵn.
- ◇ **Q-Learning:** Thuật toán kinh điển nhất của RL Chủ động, cho phép AI tự lần mò học hàm Giá trị mà không cần hiểu mô hình vật lý của thế giới.
- ◇ **DRL (Deep Reinforcement Learning):** Sự kết hợp vô tiền khoáng hậu giữa RL và Deep Learning giúp AI giải quyết không gian trạng thái vô hạn.
- ◇ **Lợi ích cốt lõi:** RL là bước đi tự nhiên nhất để biến một cỗ máy tính toán thành một "Thực thể" có mục đích, có khả năng sinh tồn và học hỏi.